Concordia University
COMP 249 – Winter 2026
Object-Oriented Programming II - Assignment # 3
Due 11:59 PM – Wednesday, April 07th, 2026

Warning: Redistribution or publication of this document or its text, by any means, is strictly prohibited. Additionally, publishing the solution publicly, at any point of time, will result in an immediate filing of an academic misconduct.

Purpose: The purpose of this assignment is help you practice Interfaces, Generics, ArrayList, and Linked Lists.

IMPORTANT NOTE: IN THIS ASSIGNMENT, YOU ARE **NOT PERMITTED** TO USE ANY OF THE JAVA BUILT-IN CLASSES not explicitly allowed in the assignment document. Using any of these will result in zero marks! In other words, you need to code whatever is needed by yourself!!! Of course, you can use the classes String, Scanner ArrayList and LinkedList.

General Guidelines When Writing Programs:

- Include the following comments at the top of your source codes

  // ----------

  // Assignment (include number)

  // Question: (include question/part number, if applicable)

  // Written by: (include your name and student id)

  // ----------

- In a comment, give a general explanation of what your program does. As the programming questions get more complex, the explanations will get lengthier.
- Include comments in your program describing the main steps in your program.
- Display a welcome message which includes your name(s).
- Display clear prompts for users when you are expecting the user to enter data from the keyboard.
- All output should be displayed with clear messages and in an easy-to-read format.
- End your program with a closing message so that the user knows that the program has terminated.

Redistribution or publication of this document or its text, by any means, is strictly prohibited.

Learning Objectives

Transform your A2 SmartTravel system into a professional Java application by mastering:

- **Collections**: Replace arrays with dynamic ArrayList & LinkedList for flexible data management.
- **Interfaces**: Define shared behaviors (Identifiable, Billable, CsvPersistable, and Comparable<T>) across all classes.
- **Generics**: Build reusable components (Repository<T>, GenericFileManager<T>, RecentList<T>) that will work with any data type
- **Filtering**: Use Predicate<T> functional interfaces for powerful, reusable search operations
- **Smart Sorting**: Implement meaningful natural ordering in every class using Comparable<T>

Assignment Overview

Start from your working A2 submission - all the existing menus (1-6, 8-11), file I/O, and dashboard functionality must remain 100% operational. A3 builds new capabilities **on the top** of A2 without breaking anything.

Big Picture Goals:

1. Replace rigid arrays with flexible collections
2. Factor common behaviors into interfaces
3. Create generic utilities that work with **any** SmartTravel entity
4. Add powerful filtering and business-smart sorting
5. Replace 4 specific file managers with 1 universal generic manager

Part 1: Collections Foundation (25 marks)

Replace Arrays with Lists (15 Marks)

Convert **all** fixed-size arrays in SmartTravelService to dynamic collections:

Transformations:

- Client[] clients + clientCount → List<Client> clients = new ArrayList<>()
- Update **each service method** (addClient, getClient(i), getClientCount, etc.) to use List operations
  - Replace clientCount → clients.size()
  - Replace array indexing → clients.get(i)
  - Replace manual counting → clients.add(client)

Redistribution or publication of this document or its text, by any means, is strictly prohibited.

Expected benefits students will experience:

- No more ArrayIndexOutOfBoundsException
- Automatic size tracking
- Cleaner enhanced for-loops: for(Client c : clients)

LinkedList Use Case : RecentList<T> Generic Wrapper

Real applications track "recently viewed" items across multiple entity types. Create a **simple generic wrapper class** RecentList<T> that that internally uses **Java's built-in java.util.LinkedList<T>**. This wrapper will manage a fixed-size history of recently viewed items (for example, trips), always keeping the most recent items at the front.

Implementation Requirements

service/RecentList.java – You must implement this **complete class**:

Core functionality:

- **Generic:** Works with Trip, Client, Accommodation, etc.
- **addRecent(T item):** Add to **front** (addFirst), auto-remove oldest from **end** if over capacity (max 10 items)
- **printRecent(int maxToShow):** Display up to N most recent items (most recent first)
- **Capacity management:** Fixed maximum of 10 items

```
public class RecentList<T> {
    private LinkedList<T> list = new LinkedList<>();
    private final int MAX_SIZE = 10;
    public void addRecent(T item) { }
    public void printRecent(int maxToShow) { }
    public int size() { }
    public boolean isEmpty() { }
}
```

README requirement: Explain why LinkedList vs ArrayList for this use case.

Part 2: Professional Interfaces (30 marks)

Interface Specifications

Create four interfaces in interfaces/ package:

1. **Identifiable** (Client, Trip, Accommodation, Transportation)

```
package interfaces;
public interface Identifiable {
    String getId();  // Returns "C1001", "T2001", "A4001", etc.
}
```


Redistribution or publication of this document or its text, by any means, is strictly prohibited.

2. Billable (Only Trip implements (use existing pricing logic))

```
package interfaces;
public interface Billable {
    double getBasePrice();
    double getTotalCost();  // Includes accommodations + transportation
}
```

3. CsvPersistable (Client, Trip, Transportation, Accommodation)

```
package interfaces;
public interface CsvPersistable {
    // CSV string matching A2 format. ("C1001;Sophia;Rossi;sophia@example.com")
    String toCsvRow();
}
```

Interface Implementation Requirements

ALL model classes implement appropriate interfaces:

- Client → Identifiable, CsvPersistable, Comparable<Client>
- Trip → Identifiable, Billable, CsvPersistable, Comparable<Trip>
- Accommodation → Identifiable, CsvPersistable, Comparable<Accommodation>
- Transportation → Identifiable, CsvPersistable, Comparable<Transportation>

Each CsvPersistable class provides:

public static Client fromCsvRow(String csvLine) **throws** InvalidClientDataException

Parses one CSV row into a fully-formed object with format validation.

4. Business-Natural Ordering

All classes implement Comparable<T> with class-specific business priority:

Each class defines its **natural business priority** via compareTo():

Class	Natural Sort Order	Business Rationale
Client	Total spent (descending)	Most valuable customers first
Trip	Total cost (descending)	Highest revenue trips first
Accommodation	Price per night (descending)	Luxury options first
Transportation	Base price (descending)	Premium transport first

Redistribution or publication of this document or its text, by any means, is strictly prohibited.

Part 3: Generic Repository Pattern (20 marks)

Repository Role

Raw storage (List<Client>) handles CRUD. **Repository<T>** handles **queries**:

- clientReo.add(c) // Storage
- clientRepo.findById(id) // Query
- clientRepo.filter(p) // Advanced search

Service mirroring clarification:

```
public void addClient(Client c) {
   clients.add(c);          // Primary storage
   clientRepo.add(c);    // Mirror copy for queries
}
```

Required Repository Methods

```
public class Repository<T extends Identifiable & Comparable<? super T>> {

    // 1. Add items (mirrors main List)
    public void add(T item);

    // 2. ID-based lookup (throws EntityNotFoundException)
    public T findById(String id);

    // 3. FILTERING: Accepts any Predicate<T> "yes/no question"
    public List<T> filter(Predicate<T> predicate);

    // 4. Smart sort using business natural order
    public List<T> getSorted();

}
```

Predicate Filtering Examples

Predicate<T> = any "yes/no question" about type T: (**import** java.util.function.Predicate)

[Type] [name] = item -> [true/false condition about item]

Predicate<Trip> expensiveTrips = trip -> trip.getTotalCost() > 2000

tripRepo.filter(expensiveTrips)

Part 4: Generic File Management (15 marks)

Single GenericFileManager Replaces 4 Specific Managers

A2: ClientFileManager, TripFileManager, AccommodationFileManager, TransportationFileManager

A3: GenericFileManager<T extends CsvPersistable> handles **all types**:

clients = GenericFileManager.load("clients.csv", Client.class);

trips = GenericFileManager.load("trips.csv", Trip.class);

GenericFileManager.save(clients, "clients.csv");

Redistribution or publication of this document or its text, by any means, is strictly prohibited.

Required Methods

public static <T **extends** CsvPersistable> List<T> load(String filepath, Class<T> clazz);

public static <T **extends** CsvPersistable> **void** save(List<T> items, String filepath);

Implementation approaches (student choice):

1. Switch expressions on clazz.getSimpleName()
2. if/else chain checking class names

Backward Compatibility

A2 menus must continue working unchanged. Options:

- Keep A2 FileManagers as fallback
- Add boolean flag useGenericPersistence
- Overload loadAllData() methods

Part 5: Advanced Analytics Menu (10 marks)

Extend existing A2 menu by updating Menu 7 while keeping all other A2 functionality unchanged:

=== SmartTravel A3 Console ===

1-6. [Keep A1/A2 menus exactly as-is]

7. Advanced Analytics ← **NEW A3 SECTION**

```
    ├── 7.1 Trips by Destination (Predicate filter)
    ├── 7.2 Trips by Cost Range (Predicate range)
    ├── 7.3 Top Clients by Spending (RecentList + natural sort)
    ├── 7.4 Recent Trips (RecentList demo)
    ├── 7. 5 Smart Sort Collections (business natural order)
    └── 7.6 Back to main menu
```

8-11. [Keep A2 menus unchanged]

0. Exit

Redistribution or publication of this document or its text, by any means, is strictly prohibited.

Submitting Assignment 3

WHAT TO SUBMIT:

You must submit a single ZIP file: **A3-YourStudentID.zip**

A3-YourStudentID/

```
├── src/          (ALL .java files)
├── data/         (your CSV test files)
├── output/       (generated dashboard)
└── README.txt    (confirm A2 compatibility + LinkedList justification)
```

README.txt must confirm:

A2 Compatibility: [Y/N - all original menus work unchanged]

LinkedList Justification: [1-2 sentences why LinkedList for recentTrips]

IMPORTANT: make sure you submit way before the closing date to avoid "surprises". Please also note that we cannot submit it for you as the submission must come from your account. We will not accept any submission by email for whatever reason. Excuses like "my system crashed when I was submitting and when it restarted it was past the deadline by 2 minutes", "I thought my friend will submit but they did not", "I forgot to submit", "I suddenly slept and woke up 3 minutes after the closing time", or similar will not be accepted and will go **unanswered**.

IMPORTANT (Please read very carefully): Additionally, which is very important, a demo will take place with the markers afterwards. Markers will inform you about the details of demo time and how to book a time slot for your demo. ***If working in a group, both members must be present during demo*** time. ***Different marks may be assigned to teammates based on this demo***.

If you ***fail to demo***, a ***zero mark is assigned*** regardless of your submission. If you book a demo time, and ***do not show up***, for whatever reason, you will be allowed to reschedule a second demo but a **penalty of 50%** will be applied.***Failing to demo at the second appointment*** will result in **zero marks** and ***no more chances*** will be given under any conditions.

Evaluation Criteria for Assignment 3 (10 points)

Total	**10 pts**
Lists + RecentList<T>	2.5 pts
4 Interfaces + factories	3 pts
Repository<T> + Predicate	2 pts
GenericFileManager	1.5 pts
Advanced Menu 7	1 pt

Redistribution or publication of this document or its text, by any means, is strictly prohibited.